



Plano de Aula

Sistema de Registro de Visitantes com JavaScript

Duração: 150 minutos (2h30)



Informações Gerais

Disciplina: Linguagem JavaScript

Tema: Desenvolvimento de aplicação web com JavaScript

Objetivo Geral: Construir um sistema funcional de registro de visitantes integrando conceitos de arrays, objetos, funções e manipulação DOM

Metodologia: Aula prática com desenvolvimento guiado em 4 etapas progressivas



Estrutura da Aula

0-15 min **Abertura e Contextualização**

Objetivos:

- Apresentar o projeto e sua relevância prática
- Motivar os alunos com aplicação real
- Revisar brevemente conceitos essenciais

Atividades:

1. Apresentação do cenário (5 min)

- Demonstrar o sistema funcionando (versão pronta)
- Explicar o contexto: portarias de prédios comerciais precisam registrar visitantes, controlar acessos e gerar relatórios

2. Visão geral técnica (5 min)

- Mostrar arquitetura: dados → funções → interface
- Explicar que construirão o sistema do zero em etapas

3. Revisão rápida (5 min)

- Arrays e objetos: como armazenam dados
- Funções: reutilização de código
- DOM: conexão HTML-JavaScript

15-45 min Etapa 1: Modelagem de Dados

Objetivo: Estruturar dados usando arrays e objetos literais

Explicação Teórica (10 minutos)

Apresentar no quadro/projetor os conceitos fundamentais:

```
// Objetos literais representam entidades const visitante  
= { nome: "João Silva", documento: "123.456.789-00",  
  empresa: "Tech Solutions" }; // Arrays armazenam coleções  
const visitantes = [visitante1, visitante2, visitante3];
```

Conceitos-chave:

- Objetos: propriedades e valores (pares chave-valor)
- Arrays: lista ordenada de elementos
- Estruturas aninhadas: objetos dentro de arrays

Atividade Prática (20 minutos)

Tarefa: Criar a estrutura de dados do sistema

Instruções:

1. Criar arquivo dados.js
2. Definir array de proprietários de salas
3. Criar array vazio para registros de visitas
4. Planejar estrutura de um registro de visita (discutir em conjunto)

```
const proprietarios = [ { id: 1, nome: "Dr. Carlos  
Mendes", sala: "101", especialidade: "Advocacia" }, { id:  
2, nome: "Dra. Ana Costa", sala: "205", especialidade:  
"Contabilidade" }, { id: 3, nome: "Eng. Pedro Santos",  
sala: "310", especialidade: "Engenharia" } ]; const  
registros = []; // Modelo de registro { id: 1, visitante:  
"Maria Oliveira", documento: "987.654.321-00",
```

```
proprietarioId: 2, data: "2025-10-28", horaEntrada:  
"14:30", horaSaida: null, motivo: "Reunião contábil" }
```

 **Dica:** Verificar se todos os alunos criaram as estruturas corretamente antes de avançar

45-85 min Etapa 2: Funções de Lógica de Negócio

Objetivo: Implementar funções para manipular dados

Explicação Teórica (10 minutos)

Revisar conceitos de funções:

- Funções como blocos reutilizáveis
- Parâmetros e retorno
- Manipulação de arrays: `push()` , `filter()` , `find()`

Atividade Prática (30 minutos)

Tarefa: Criar arquivo `funcoes.js` com as seguintes funções:

Função	Tempo	Descrição
<code>registrarEntrada()</code>	10 min	Cria novo registro de visita com data/hora atual
<code>registrarSaida()</code>	8 min	Atualiza registro existente com horário de saída
<code>relatorioPorData()</code>	6 min	Filtra registros por data específica
<code>relatorioPorProprietario()</code>	6 min	Filtra registros por ID do proprietário

 **Dica:** Construir cada função junto com os alunos, explicando linha por linha. Testar no console do navegador após cada implementação.

Exemplo de Implementação:

```
function registrarEntrada(nomeVisitante, documento,
proprietarioId, motivo) { const novoRegistro = { id:
registros.length + 1, visitante: nomeVisitante, documento:
documento, proprietarioId: proprietarioId, data: new
Date().toISOString().split('T')[0], horaEntrada: new
Date().toLocaleTimeString().slice(0, 5), horaSaida: null,
motivo: motivo }; registros.push(novoRegistro); return
novoRegistro; }
```

85-95 min  **Intervalo****95-140 min** **Etapa 3: Interface HTML e Integração**

Objetivo: Criar interface funcional conectada às funções JavaScript

Explicação Teórica (10 minutos)

Demonstrar conceitos de integração:

- Estrutura HTML básica
- Eventos: onclick
- Manipulação DOM: getElementById() , innerHTML
- Captura de valores: value

```
// Exemplo de integração
document.getElementById('botao').onclick = function() {
  const valor = document.getElementById('input').value;
  document.getElementById('resultado').innerHTML = valor; };

```

Atividade Prática (35 minutos)

Tarefa: Criar interface completa com três arquivos:

1. **index.html** - Estrutura da interface com:

- Seção de registro de entrada
- Seção de registro de saída
- Seção de relatórios
- Área de exibição de resultados

2. **interface.js** - Funções de integração:

- preencherSelects() - preenche dropdowns
- registrarNovaEntrada() - captura dados do formulário
- registrarNovaSaida() - processa saída

- gerarRelatorioPorData() - gera relatório por data
- gerarRelatorioPorProprietario() - gera relatório por proprietário
- exibirRelatorio() - exibe dados em tabela

3. **CSS inline** - Estilização básica e responsiva

 **Dica:** Enfatizar a importância de validar dados antes de processar e fornecer feedback visual ao usuário

140-150 min Testes e Demonstrações**Atividades (10 minutos):**

1. **Testes individuais:** Cada aluno testa seu sistema
 - Registrar 2-3 entradas
 - Registrar 1 saída
 - Gerar relatórios por data e por proprietário
2. **Suporte:** Professor circula auxiliando
3. **Demonstrações:** Voluntários apresentam para turma

150-160 min Fechamento e Discussão**Reflexões Guiadas:****1. Revisão dos conceitos:**

- Como arrays e objetos organizaram nossos dados?
- Por que separar funções ajuda na manutenção?
- Como HTML e JavaScript se comunicam?

2. Melhorias possíveis:

- Validação mais robusta de dados
- Persistência de dados (localStorage)
- Upload de fotos dos visitantes
- Sistema de notificações para proprietários
- Dashboard com estatísticas visuais

3. Aplicações reais:

- Onde mais esse tipo de sistema é usado?
- Que adaptações seriam necessárias para outros contextos?

**Tarefa para Casa (Opcional)**

- Adicionar campo "empresa" do visitante
- Criar função de busca por nome de visitante
- Melhorar CSS da interface com cores personalizadas



Materiais Necessários



Hardware

- Computadores com navegador web moderno
- Projetor para demonstrações



Software

- Editor de código (VS Code, Sublime, etc.)
- Navegador com DevTools (Chrome, Firefox)



Materiais de Apoio

- Slides com conceitos principais
- Código de exemplo pronto (backup)



Documentação

- Folha de referência JavaScript
- Guia de métodos de array



Avaliação

Critérios de Avaliação:



Estruturas de dados corretas e bem organizadas



Funções implementadas e funcionais



Interface conectada às funções



Sistema registra dados corretamente



Sistema exibe informações adequadamente



Código limpo e comentado

Níveis de Desempenho:

Nível	Descrição
-------	-----------

Excelente	Sistema completo, funcional, com código organizado e possíveis melhorias implementadas
Bom	Sistema funcional com todas as etapas implementadas corretamente
Satisfatório	Sistema funciona parcialmente, algumas funcionalidades precisam ajustes
Insuficiente	Sistema não funciona ou possui erros críticos nas etapas principais



Dicas para o Professor



Dica: Durante as atividades práticas:

Circule pela sala identificando dificuldades comuns. Agrupe alunos com problemas similares para explicações conjuntas.



Dica: Para debugging:

Use o console do navegador (F12) para debugar junto com os alunos. Ensine-os a ler mensagens de erro.



Dica: Motivação:

Celebre pequenas vitórias - quando uma função funcionar pela primeira vez, reconheça o progresso do aluno.



Dica: Gestão do tempo:

Tenha código backup pronto caso algum aluno perca muito tempo em erros. Adapte o ritmo conforme necessidade da turma.



Dica: Diferenciação:

Para alunos mais avançados, sugira desafios extras

(validações complexas, funcionalidades adicionais). Para alunos com dificuldades, foque nas etapas essenciais.

Recursos Complementares

- **MDN Web Docs:** Documentação completa de JavaScript
- **JavaScript.info:** Tutorial moderno e detalhado
- **DevTools:** Guia de uso das ferramentas de desenvolvedor
- **W3Schools:** Exemplos práticos e interativos

Checklist Final

Antes da aula, verificar:

- ☐ Computadores funcionando com navegadores atualizados
- ☐ Projetor testado
- ☐ Código de exemplo backup preparado
- ☐ Material de apoio impresso (se necessário)
- ☐ DevTools funcionando nos computadores
- ☐ Ambiente de desenvolvimento configurado

Documento gerado em: 28 de outubro de 2025

 Boa aula!